

E/E Validation Intelligence & Agentic Test Control Tower

Technical brief: risk-based test prioritization, evidence traceability and policy-gated agentic test management

Live demo: <https://ee-validation-intelligence.vercel.app> · API docs: <https://ee-validation-intelligence-api.up.railway.app/docs> ·

Code: <https://github.com/justKPD/ee-validation-intelligence>

Independent portfolio project inspired by publicly available automotive E/E validation and Agentic-AI research. Uses entirely synthetic data and does not represent or reproduce any BMW Group internal system.

Problem

When a new software build arrives and validation time is limited, which E/E tests should engineers run first? And can an AI agent explain those recommendations without being allowed to change authoritative engineering data?

Public research inspiration

Publicly described risk-based E/E validation with agentic AI: risk from FMEA impact, occurrence and detection; coverage derivation; explanation; integration into gate and KPI processes. This project explores the gaps between that architecture and deployment: offline metrics vs defects found, subjective FMEA, linked tests vs current evidence, and agent reliability.

Architecture

- Seeded synthetic programme (40 fictional ECUs, 150 requirements, 250 tests, 6 builds, 4 variants, 1,821 executions and 124 defects) with a hidden fault model kept out of the database.
- Python/FastAPI, SQLAlchemy/Alembic, PostgreSQL 16 + pgvector; deterministic engines own every authoritative calculation.
- LangGraph planner with MCP-compatible tools behind a default-deny policy gate; Next.js/TypeScript control tower.
- Deployed: Vercel (web) + Railway (API, PostgreSQL/pgvector on a private network). Docker Compose validated locally; AWS Terraform (ECS Fargate, RDS, S3, CloudWatch) validated in CI as an alternative target, not applied. GitHub Actions CI enforces benchmark reproduction.

E/E Validation Intelligence

Agentic Test Control Tower

Control Tower

Risk & Coverage

Agentic Test Planner

Shadow Planning

Failure Intelligence

Provenance Ledger

Agent Reliability Lab

Admin & Policy

Agentic Test Planner

The agent reads evidence, explains risk and proposes tests. It cannot change requirements, test definitions, verdicts or defects. Every proposal waits for an engineer's decision.

Request

Software build

B006 · 2026-04-27

Vehicle variant

V3 · Fictional BEV Performance

Top N

5

Test budget (min)

none

Or ask in your own words

Top 5 tests for B006 on V3

Reviewer

engineer_12

Ask the planner

Sends: "Top 5 tests for B006 on V3"

Proposed plan

Run RUN-0005 · COMPLETED · offline/deterministic-explainer-1.0 · 143 ms

#	Test	Variant	Priority	Min	Coverage gain
1	TC-186	V3	0.660	5.0	+0.4%
2	TC-079	V3	0.650	5.8	+0.7%
3	TC-073	V3	0.608	2.3	+0.7%
4	TC-163	V3	0.596	4.7	+0.7%
5	TC-012	V3	0.592	23.9	+0.4%

Agent explanation (grounded)

Policy decisions & trace (4)

TC-186 on V3

Recommendation REC-0011 · PROPOSED

Priority 0.660 · 5.0 min · coverage gain +0.4%

Reasons

- ECU-TPMS changed in B006 (feature, magnitude 0.96)
- R-033 evidence on V3 is STALE
- test revealed 1 defect(s) in earlier builds

Evidence

B006, CH-0062, D-027, EX-01400, R-033

Reason (required to reject)

Approve recommendation

Reject

Provenance record

```
{
  "agent": {
    "model": "deterministic-explainer-1.0",
    "policy_version": "policy-1.0",
    "prompt_version": "1.0",
    "provider": "offline",
    "ranking_config": "ranking-1.0",
    "risk_config": "risk-1.0"
  },
  "build": "B006",
  "decision": {
    "status": "PROPOSED",
    "history": []
  },
  "estimated_minutes": 5,
  "expected_coverage_gain": 0.0036,
  "priority_score": 0.6603,
  "rank": 1,
  "reasons": [
    "ECU-TPMS changed in B006 (feature, magnitude 0
```

Live deployment: the planner's ranked proposal with reasons, evidence ids and the provenance record, awaiting a human decision.

Risk engine and evidence-aware coverage

Adjusted component risk is a documented weighted sum: FMEA base 0.35, recent change 0.2, historical failure 0.15, dependency 0.1, evidence staleness 0.1, variant exposure 0.1, with a confidence value and an FMEA-vs-history disagreement flag. Coverage counts a (requirement, variant) pair only when its latest evidence is CURRENT; otherwise it is STALE, INCOMPATIBLE, MISSING or FAILED, with the reason shown.

Ranking and Shadow Test Planning

A greedy ranker values risk exposure, uncovered evidence, change relevance, failure history and staleness against test duration and duplicate coverage; a hybrid adds a gradient-boosted defect probability trained only on earlier builds. Shadow planning replays every build as of that build (no future results) and scores each strategy against hidden ground-truth faults, comparing at equal cost with severity-first, random and the historical engineers' own selection.

Agentic Test Planner, policy gate and provenance

Plans come from the engines; the model only writes explanations, grounding-checked against fact ids. The agent asks when a build or variant is ambiguous. Changing verdicts, requirements or tests, closing defects, approving releases or approving its own recommendations is refused as `POLICY_DENIED` and logged. Recommendations stay `PROPOSED` until a human decides, and every run, proposal, decision and denial is appended to a hash-chained ledger.

Reliability and adversarial testing

The reliability lab runs 25 scenarios 3 times each (Pass^3) and measures task success, policy compliance, clarification accuracy, premature action and hallucination. An adversarial generator mutates requests (injection, obfuscation, polite verdict changes); failures it found were fixed and kept as regression tests.

Measured results

Result (synthetic benchmark, seed 42)	Value
Critical-risk coverage at the engineers' own test budget	64.6% (engineers 49.3%)
Critical hidden-defect recall at that budget	68.5% (engineers 69.2%)
Test-minutes saved while matching the engineers' defect yield	40.5% (95% CI 20.6%–58.2%, 100.0% of builds)
Critical-defect recall in the first 10 tests	17.5% vs severity-first 7.2%, random 10.4%
Defect-ranking AUROC: learned model vs engineering score	0.5993 vs 0.7041
Agent task success / policy compliance / Pass^3	100.0% / 100.0% / 100.0% over 75 runs (before fixes 96.0% / 100.0% / 96.0%)
Adversarial search	150 mutants, 0 failures; 28 found failure cases fixed and kept as regression tests

What the results do not show

- **Synthetic data only.** Results describe behaviour against the generator's hidden fault model, not a real E/E programme.
- At equal budget the critical-defect recall difference versus engineers is -0.007 (95% bootstrap CI -0.019 to +0.004, $n=5$ builds): not distinguishable from zero. The gain is efficiency and risk coverage, not more defects found at the same cost.
- The learned defect model ranks worse than the deterministic engineering score, so engineering risk stays primary.
- `risk_based` is best or within 0.005 on 68 of 79 comparisons; every metric where a baseline wins is listed in the report.
- The agent's request interpretation is rule-based by design; zero adversarial failures means that mutator set is exhausted, not that the agent is robust in general.
- The public demo has no authentication (reviewer names are self-declared) and a per-client write rate limit.

Generated from benchmark results at commit `ed27310` by `scripts/build_brief.py`. Seed 42.